

## Módulo 2 — Prompt Engineering Profesional

---

# Capítulo 22 — Proyecto Integrador del Módulo 2

---

## | Sección 01 — Presentación del Proyecto Integrador

## Objetivos de aprendizaje

---

- Integrar todos los conceptos desarrollados durante el módulo.
- Diseñar una solución completa desde la perspectiva del AI Engineering.
- Aplicar un proceso profesional de análisis, diseño, implementación y evaluación.
- Preparar la transición hacia el estudio de los modelos fundacionales.

# Introducción

---

Los capítulos anteriores abordaron cada componente de forma individual: Ingeniería del Prompt, patrones de Prompt Engineering, Prompt Engineering para producción, ingeniería conversacional, arquitecturas basadas en prompts y laboratorios prácticos.

Este proyecto integrador propone combinar todos esos conocimientos para resolver un caso de negocio de principio a fin. El objetivo no consiste en construir un prompt aislado, sino en diseñar una solución completa que pueda crecer, mantenerse y evaluarse con criterios de ingeniería.

## El desafío

---

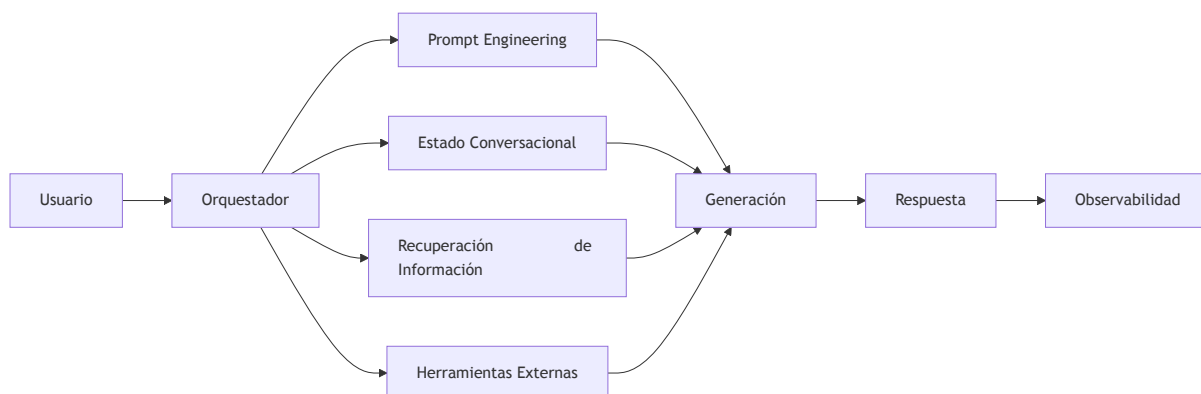
Una organización desea implementar un **Asistente Corporativo Inteligente** basado en un Large Language Model (LLM). El sistema deberá:

- responder consultas internas;
- recuperar documentación institucional;
- clasificar solicitudes;
- generar respuestas estructuradas;
- registrar incidentes cuando corresponda;
- mantener conversaciones de larga duración;
- integrarse con sistemas existentes.

El proyecto deberá diseñarse como si fuera a evolucionar hasta convertirse en un sistema de producción.

## Alcance del proyecto

El siguiente diagrama representa el alcance conceptual del proyecto: los grandes bloques de responsabilidad que se trabajarán a lo largo de las secciones.



Este diagrama no describe la arquitectura técnica interna — eso corresponde a la Sección 03. Aquí el foco está en identificar qué capacidades integra el sistema y cómo se relacionan a nivel conceptual.

La arquitectura propuesta constituye un punto de partida que podrá ampliarse en módulos posteriores.

# Entregables esperados

| Entregable                | Objetivo                                      |
|---------------------------|-----------------------------------------------|
| Análisis del problema     | Comprender el dominio de negocio.             |
| Arquitectura              | Definir componentes y responsabilidades.      |
| Diseño de prompts         | Especificar prompts reutilizables.            |
| Estrategia conversacional | Estado, contexto y memoria.                   |
| Plan de evaluación        | Casos de prueba y métricas.                   |
| Estrategia de despliegue  | Versionado, observabilidad y mejora continua. |

## Caso de estudio motivador

---

Un equipo construye una primera versión del asistente con un único prompt. El prototipo funciona durante las demostraciones, pero incorporar nuevas funcionalidades se vuelve cada vez más costoso.

Tras aplicar los principios estudiados en este módulo, el equipo divide responsabilidades, incorpora un orquestador, define prompts especializados, agrega evaluación continua y establece un proceso de PromptOps. La nueva arquitectura requiere un mayor esfuerzo inicial, pero reduce significativamente el costo de evolución del sistema.

Este recorrido —del prototipo al sistema sostenible— es el que este capítulo propone transitar.

## Cómo leer este capítulo

---

Las secciones siguientes avanzan en secuencia:

- **Sección 02** define el análisis del problema y el dominio de negocio.
- **Sección 03** especifica la arquitectura técnica del sistema.
- **Sección 04** trabaja el diseño de prompts y la estrategia conversacional.
- **Sección 05** aborda el plan de evaluación y las métricas de éxito.
- **Sección 06** describe la estrategia de despliegue y operación continua.
- **Sección 07** cierra el módulo con la síntesis de aprendizajes y la transición al Módulo 3.

Cada sección puede leerse de forma independiente como referencia, pero el proyecto cobra sentido completo al recorrerlas en orden.

---

## Módulo 2 — Prompt Engineering Profesional

---

# Capítulo 22 — Proyecto Integrador del Módulo 2

---

## Sección 02 — Análisis del Problema

*"Todo proyecto exitoso comienza comprendiendo el problema antes de elegir la tecnología."*

### Objetivos de aprendizaje

- Realizar el análisis inicial de un proyecto de AI Engineering.
  - Identificar actores, objetivos y restricciones.
  - Traducir necesidades de negocio en requerimientos técnicos.
  - Definir el alcance de una primera versión viable.
- 

### Introducción

Antes de escribir un prompt, seleccionar un modelo o definir una arquitectura, un AI Engineer necesita comprender el problema que intenta resolver.

Las decisiones tomadas durante esta etapa condicionarán el resto del proyecto. Un análisis incompleto suele producir soluciones técnicamente interesantes, pero alejadas de las necesidades reales del negocio.

Esta sección desarrolla el primer entregable del proyecto integrador: el análisis del problema.

---

### Paso 1: Comprender el problema

El análisis inicial debe responder preguntas como:

- ¿Qué proceso desea mejorar la organización?

- ¿Quién utilizará el sistema?
- ¿Qué tareas serán asistidas por el Large Language Model (LLM)?
- ¿Qué tareas seguirán siendo responsabilidad de las personas?
- ¿Cómo se medirá el éxito del proyecto?

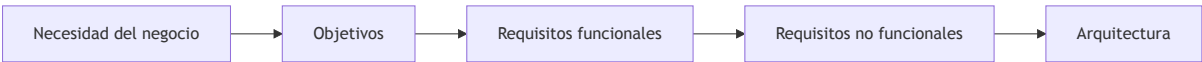
Estas respuestas constituyen la base del diseño posterior.

Identificación de actores

| Actor                 | Responsabilidad                 |
|-----------------------|---------------------------------|
| Usuario final         | Interactúa con el asistente.    |
| Responsable funcional | Define reglas de negocio.       |
| Equipo de IA          | Diseña la solución.             |
| Equipo de IT          | Integra la plataforma.          |
| Administrador         | Supervisa operación y métricas. |

Cada actor posee expectativas y necesidades diferentes. Identificarlos desde el inicio evita que el diseño técnico responda a supuestos incorrectos sobre quién usa el sistema y con qué propósito.

Del problema a los requisitos



El objetivo no consiste únicamente en automatizar tareas, sino en construir una solución sostenible. Cada paso del diagrama transforma una necesidad expresada en lenguaje de negocio en un elemento técnico concreto y verificable.

Caso de estudio

Una empresa desea implementar un asistente para su mesa de ayuda.

Tras las primeras reuniones, el equipo observa que los agentes invierten más tiempo clasificando y registrando cada incidente que respondiendo la consulta en sí. Al revisar los registros existentes y preguntar a los agentes sobre su carga de trabajo diaria, el equipo comprende que el cuello de botella no está en la calidad de las respuestas sino en la fricción del proceso administrativo.

Esta conclusión modifica completamente el alcance del proyecto: la prioridad deja de ser la conversación y pasa a ser la automatización del proceso de atención. El caso ilustra un principio central del análisis: lo que la organización cree que necesita y lo que realmente necesita no siempre coinciden. La diferencia solo se revela al explorar el problema con profundidad.

---

### Actividades propuestas

1. Describir el problema de negocio en términos concretos, evitando referencias a tecnologías específicas.
2. Identificar actores y responsabilidades, y documentar el resultado como un activo verificable del proyecto.
3. Definir objetivos medibles que permitan evaluar el éxito al finalizar la primera versión.
4. Delimitar el alcance de la primera versión: qué queda dentro y qué queda fuera.
5. Registrar supuestos, riesgos y restricciones conocidas como documentación base para las etapas siguientes.

---

### Buenas prácticas

- Comprender el negocio antes de diseñar la solución técnica.
- Validar el alcance con los interesados antes de avanzar.
- Documentar supuestos y restricciones específicos del problema que se está analizando.
- Priorizar objetivos medibles sobre objetivos cualitativos.

---

### Errores frecuentes

- Comenzar escribiendo prompts sin haber completado el análisis previo.
- Confundir deseos con requisitos.

- Intentar resolver todos los problemas en la primera versión.
  - No definir criterios de éxito antes de comenzar el desarrollo.
- 

### **Ideas clave**

- Todo proyecto comienza con el análisis del problema, no con la elección de la tecnología.
  - La tecnología debe responder a objetivos del negocio, no al revés.
  - Un buen análisis reduce riesgos durante el desarrollo y evita rediseños costosos.
- 

Con el problema comprendido, los actores identificados y el alcance delimitado, el siguiente paso es traducir ese análisis en una arquitectura concreta. En la próxima sección diseñaremos la arquitectura de referencia del proyecto, definiendo componentes, responsabilidades y flujos de interacción entre ellos.

---

## Módulo 2 — Prompt Engineering Profesional

---

# Capítulo 22 — Proyecto Integrador del Módulo 2

---

## Sección 03 — Arquitectura de Referencia

*"Una arquitectura no describe únicamente cómo funciona un sistema. Describe cómo podrá evolucionar durante los próximos años."*

### Objetivos de aprendizaje

- Diseñar una arquitectura de referencia para el proyecto integrador.
- Identificar los principales componentes de una solución basada en Large Language Model (LLM).
- Definir responsabilidades y relaciones entre los distintos módulos.
- Aplicar los principios de desacoplamiento —la capacidad de modificar un componente sin afectar al resto del sistema— y escalabilidad estudiados durante el módulo.

### Introducción

Con el problema de negocio claramente definido, el siguiente paso consiste en diseñar la arquitectura que permitirá implementar la solución.

En esta etapa aún no se desarrollan prompts ni se elige un modelo específico. El objetivo es identificar los componentes necesarios, establecer sus responsabilidades y definir cómo colaborarán entre sí.

Una buena arquitectura permite incorporar nuevas capacidades sin rediseñar completamente el sistema.

### Componentes principales

Para este proyecto se propone una arquitectura compuesta por los siguientes elementos:

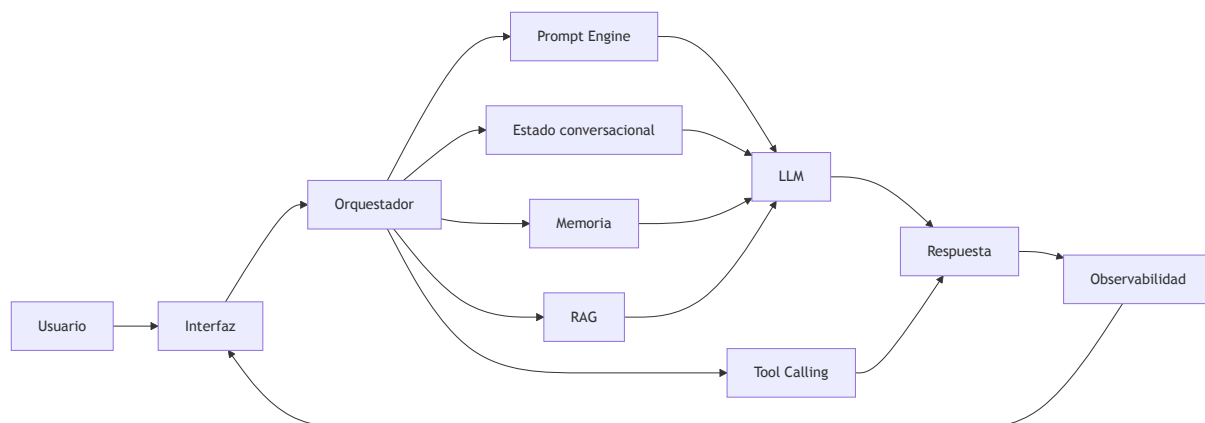
| Componente            | Responsabilidad                                                                                                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interfaz de usuario   | Recibir consultas y presentar respuestas.                                                                                                                                               |
| Orquestador           | Coordinar el flujo general del sistema: decide qué componentes activar en cada turno según la intención detectada, el estado de la sesión y la disponibilidad de información relevante. |
| Prompt Engine         | Gestionar los prompts especializados.                                                                                                                                                   |
| Estado conversacional | Mantener la continuidad de la interacción dentro de la sesión activa.                                                                                                                   |
| Memoria               | Persistir información relevante entre sesiones distintas.                                                                                                                               |
| RAG                   | Recuperar conocimiento actualizado desde fuentes documentales externas.                                                                                                                 |
| Tool Calling          | Ejecutar acciones sobre sistemas externos.                                                                                                                                              |
| Observabilidad        | Registrar métricas, eventos y auditoría.                                                                                                                                                |

El Estado conversacional y la Memoria cumplen funciones complementarias pero operacionalmente distintas: el primero sostiene el hilo de la conversación en curso; la segunda conserva lo relevante para sesiones futuras.

Cada componente posee una responsabilidad claramente delimitada. El Orquestador es el nodo central del sistema: evalúa la intención detectada por el clasificador, consulta el estado de la sesión actual y verifica si existe documentación relevante disponible antes de determinar qué componentes deben participar en la generación de la respuesta.

## Arquitectura de referencia

A diferencia del esquema conceptual presentado en la Sección 01, el siguiente diagrama detalla la arquitectura técnica del sistema con las responsabilidades diferenciadas de cada componente.



La arquitectura prioriza la separación de responsabilidades y facilita la incorporación de nuevos componentes.

## Decisiones de diseño

Durante esta etapa conviene responder preguntas como:

- ¿Qué componentes deberán ser reutilizables?
- ¿Qué información permanecerá fuera del modelo?
- ¿Dónde se administrará el estado conversacional?
- ¿Qué servicios externos participarán del proceso?
- ¿Cómo se registrarán métricas y eventos?

Responder estas preguntas antes de implementar reduce considerablemente el riesgo de rediseños posteriores. También es el momento de definir los contratos de interacción —la especificación formal de entradas y salidas esperadas de cada componente— que harán posible desarrollar y probar cada módulo de forma independiente.

## Caso de estudio

El equipo responsable del proyecto decide incorporar una base documental mediante RAG varios meses después del inicio del desarrollo.

Gracias a la arquitectura modular, el nuevo componente se integra sin modificar el resto de la solución. El Orquestador incorpora una nueva condición dentro del flujo de procesamiento: cuando detecta que la consulta requiere información documental, activa el componente RAG antes de construir el prompt y enviarlo al LLM.

La inversión realizada durante el diseño arquitectónico demuestra su valor al facilitar la evolución del sistema.

## Actividades propuestas

1. Dibujar la arquitectura del proyecto.
2. Identificar responsabilidades de cada componente.
3. Definir contratos de interacción para cada módulo.
4. Detectar posibles puntos de acoplamiento —lugares donde un cambio en un componente podría forzar cambios en otros.
5. Justificar las principales decisiones de diseño.

## Buenas prácticas

- Mantener responsabilidades bien definidas en cada componente.
- Diseñar componentes independientes que puedan modificarse sin afectar al resto.
- Favorecer la reutilización entre proyectos similares.
- Documentar todas las decisiones de diseño con su justificación en el momento en que se toman.

## Errores frecuentes

- Incorporar lógica de negocio dentro de los prompts.
- Acoplar directamente todos los componentes sin pasar por el Orquestador.
- Diseñar una arquitectura excesivamente compleja desde el inicio.
- No contemplar el crecimiento futuro del sistema.

## Ideas clave

- La arquitectura constituye el esqueleto de toda solución de AI Engineering.
- El desacoplamiento facilita el mantenimiento y la evolución del sistema.
- Diseñar correctamente desde el principio reduce costos a largo plazo.

## Transición hacia la siguiente sección

En la próxima sección comenzaremos el diseño funcional de los componentes, definiendo los prompts, los flujos conversacionales y los mecanismos de evaluación que utilizará el proyecto integrador.

---

## Módulo 2 — Prompt Engineering Profesional

---

# Capítulo 22 — Proyecto Integrador del Módulo 2

---

## Sección 04 — Diseño Funcional

*"La arquitectura define qué componentes existen. El diseño funcional define cómo colaboran para resolver el problema."*

### Objetivos de aprendizaje

- Diseñar el comportamiento funcional de la solución.
- Definir los prompts especializados y sus responsabilidades.
- Especificar el flujo conversacional del proyecto.
- Establecer una estrategia inicial de evaluación.

### Introducción

Con la arquitectura definida, el siguiente paso consiste en describir el comportamiento de cada componente.

Esta etapa conecta las necesidades del negocio con la implementación técnica.

Todavía no se escribe código. Tampoco se optimizan prompts.

El objetivo consiste en definir claramente qué hará cada componente, qué información recibirá y qué resultado deberá producir.

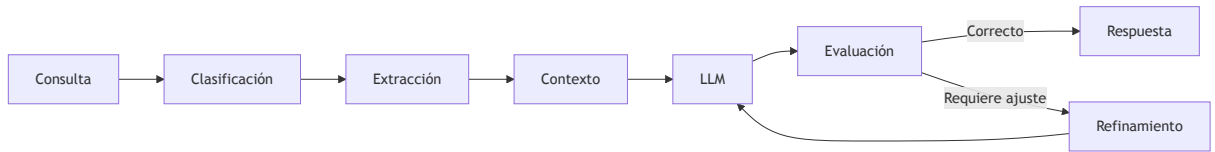
### Diseño funcional

Para este proyecto se propone dividir las responsabilidades en componentes especializados.

| Componente              | Función                                      |
|-------------------------|----------------------------------------------|
| Prompt de clasificación | Identificar la intención del usuario.        |
| Prompt de extracción    | Obtener datos relevantes de la consulta.     |
| Prompt de generación    | Construir la respuesta final.                |
| Gestor de contexto      | Preparar la información enviada al modelo.   |
| Gestor de memoria       | Recuperar información persistente.           |
| Evaluable               | Verificar calidad y formato de la respuesta. |

Cada componente puede evolucionar de manera independiente siempre que respete su contrato funcional, es decir, la especificación de qué información recibe, qué procesamiento realiza y qué resultado debe producir.

## Flujo de interacción



Este flujo permite introducir controles de calidad antes de entregar la respuesta al usuario.

## Estrategia de evaluación

Desde el comienzo del proyecto conviene definir cómo se medirá la calidad.

Algunos indicadores iniciales son:

- precisión funcional;
- consistencia del formato;
- cumplimiento de reglas de negocio;
- estabilidad entre ejecuciones;
- consumo aproximado de tokens;
- tiempo medio de respuesta.

Las métricas deberán revisarse periódicamente a medida que evolucione la solución. Estas métricas serán formalizadas y ampliadas en la siguiente sección, dedicada a la planificación de pruebas.

## Caso de estudio

Durante el diseño funcional, el equipo detecta que un mismo prompt intenta clasificar la consulta, extraer datos y generar la respuesta.

En lugar de aumentar su complejidad, divide estas responsabilidades en tres componentes independientes.

Las pruebas posteriores muestran una mejora en mantenibilidad, facilidad de depuración y reutilización.

## Actividades propuestas

1. Definir las responsabilidades de cada prompt.
2. Especificar entradas y salidas esperadas.
3. Diseñar el flujo conversacional.
4. Elaborar un primer conjunto de métricas.
5. Revisar posibles puntos de mejora antes de implementar.

## Buenas prácticas

- Mantener responsabilidades pequeñas y bien definidas.
- Diseñar contratos claros entre componentes.
- Pensar en la reutilización desde el inicio.
- Definir métricas antes de comenzar las pruebas.

## Errores frecuentes

- Crear prompts con múltiples responsabilidades.
- No documentar el comportamiento esperado.
- Posponer la evaluación para el final del proyecto.
- Diseñar componentes difíciles de reutilizar.

## Ideas clave

- El diseño funcional transforma la arquitectura en un plan de implementación.
- Los prompts deben actuar como componentes especializados.
- La evaluación comienza durante el diseño, no después del desarrollo.

## Transición hacia la siguiente sección

En la próxima sección abordaremos la planificación de las pruebas del proyecto integrador, definiendo escenarios de validación, conjuntos de evaluación y criterios de aceptación que permitan decidir objetivamente cuándo la solución está preparada para evolucionar hacia un entorno de producción. En ese contexto, el componente Evaluador definido en esta sección operará a través de los Evaluation Sets —conjuntos estructurados de casos de prueba— que se diseñarán en detalle a continuación.

---

## Módulo 2 — Prompt Engineering Profesional

---

# Capítulo 22 — Proyecto Integrador del Módulo 2

---

## Sección 05 — Estrategia de Validación

*"Una solución no está lista para producción cuando funciona una vez. Está lista cuando demuestra que puede funcionar de manera consistente."*

### Objetivos de aprendizaje

- Diseñar una estrategia de validación para una solución basada en Large Language Model (LLM).
- Definir conjuntos de evaluación (*Evaluation Sets*).
- Establecer criterios objetivos de aceptación.
- Comprender el papel de las pruebas dentro del AI Engineering.

### Introducción

En proyectos tradicionales de software es habitual verificar que una funcionalidad produzca el resultado esperado. En sistemas basados en LLM, este enfoque resulta insuficiente, y el componente Evaluador presentado en la sección anterior opera precisamente a través de los conjuntos de casos de prueba que se diseñarán en esta etapa. Las respuestas pueden variar entre ejecuciones y la calidad no depende únicamente de la ausencia de errores, sino también de la consistencia, la utilidad y el cumplimiento de restricciones.

Por este motivo, el proyecto integrador incorpora una etapa específica de planificación de pruebas antes de considerar la solución preparada para producción.

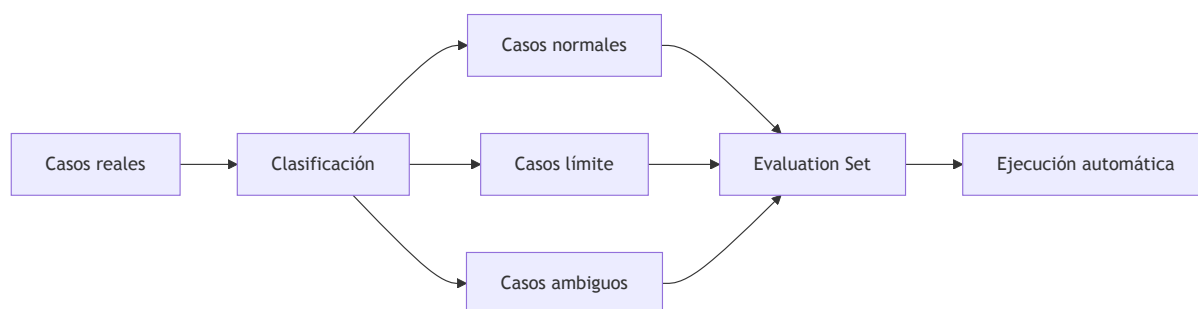
### Estrategia de validación

La validación debe contemplar distintos niveles.

| Nivel                  | Objetivo                                             |
|------------------------|------------------------------------------------------|
| Pruebas unitarias      | Validar cada componente por separado.                |
| Pruebas de integración | Verificar la interacción entre módulos.              |
| Evaluation Sets        | Comparar el comportamiento frente a casos conocidos. |
| Pruebas de usuario     | Evaluar utilidad y experiencia.                      |
| Monitoreo inicial      | Detectar desvíos después del despliegue.             |

Cada nivel aporta evidencia diferente sobre la calidad de la solución.

## Construcción de Evaluation Sets



La ejecución automática implica que cada nueva versión del sistema se compara contra el conjunto de casos sin intervención manual.

Los conjuntos de evaluación deben evolucionar junto con la aplicación. Cada incidente detectado en producción representa una oportunidad para incorporar un nuevo caso de prueba.

A continuación se muestra un ejemplo de cómo se estructura un caso de prueba en el contexto del asistente corporativo:

| Entrada                                   | Salida esperada                                                              | Criterio de aceptación                                                                   |
|-------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| ¿Cómo escalo un incidente?                | Respuesta con los pasos del proceso de escalamiento según el manual interno. | El formato debe contener al menos tres pasos numerados; no debe inventar procedimientos. |
| ¿Quién aprueba los cambios en producción? | Descripción del rol responsable de aprobación según la política interna.     | Debe mencionar el rol correcto; no debe especular con nombres propios.                   |
| ¿Cuál es el SLA para incidentes críticos? | Tiempo de respuesta definido en el contrato de nivel de servicio.            | Debe citar el valor numérico exacto; no debe aproximar ni inventar tiempos.              |

## Criterios de aceptación

Antes de avanzar a una nueva versión conviene definir indicadores mínimos. Algunos ejemplos son:

- porcentaje de respuestas correctas;
- estabilidad del formato de salida;
- cumplimiento de reglas de negocio;
- ausencia de alucinaciones críticas;
- tiempo máximo de respuesta;
- costo promedio por interacción.

Estos indicadores deben adaptarse al contexto de cada proyecto.

## Caso de estudio

Durante las pruebas del asistente corporativo, el equipo descubre que determinadas consultas ambiguas generan respuestas inconsistentes. En lugar de corregir únicamente esos ejemplos, incorpora todos los casos al *Evaluation Set* y automatiza su ejecución en cada nueva versión. Con el tiempo, el conjunto de pruebas se convierte en uno de los principales activos del proyecto.

## Actividades propuestas

1. Construir un conjunto inicial de casos de prueba.
2. Clasificar los casos por nivel de dificultad.

3. Definir criterios de aceptación cuantificables.
4. Registrar resultados de cada iteración.
5. Incorporar nuevos casos a medida que evolucione el sistema.

## Buenas prácticas

- Automatizar las pruebas siempre que sea posible.
- Mantener actualizado el *Evaluation Set*.
- Basar las decisiones en métricas.
- Revisar periódicamente los criterios de aceptación.

## Errores frecuentes

- Evaluar únicamente ejemplos favorables.
- No conservar el historial de resultados.
- Cambiar varios componentes antes de medir el impacto.
- Considerar suficientes las pruebas manuales.

## Ideas clave

- La calidad debe demostrarse mediante evidencia objetiva.
- Los *Evaluation Sets* evolucionan junto con la solución.
- Las pruebas constituyen un componente permanente del ciclo de vida del AI Engineering.

## Transición hacia la siguiente sección

En la próxima sección abordaremos la estrategia de despliegue, observabilidad y mejora continua del proyecto integrador, completando el ciclo de vida de una solución profesional basada en Inteligencia Artificial.

---

## Módulo 2 — Prompt Engineering Profesional

---

# Capítulo 22 — Proyecto Integrador del Módulo 2

---

## Sección 06 — Despliegue, Observabilidad y Mejora Continua

*"El desarrollo termina cuando se despliega. La ingeniería comienza cuando el sistema entra en producción."*

### Objetivos de aprendizaje

- Diseñar una estrategia de despliegue para una solución basada en Large Language Model (LLM).
- Incorporar observabilidad y monitoreo desde el inicio.
- Comprender el ciclo de mejora continua.
- Completar el ciclo de vida del proyecto integrador.

### Introducción

Una solución de Inteligencia Artificial no finaliza cuando supera las pruebas internas.

El verdadero desafío comienza cuando interactúa con usuarios reales, datos cambiantes y procesos críticos del negocio.

Por este motivo, el despliegue debe entenderse como una transición controlada y no como el final del proyecto.

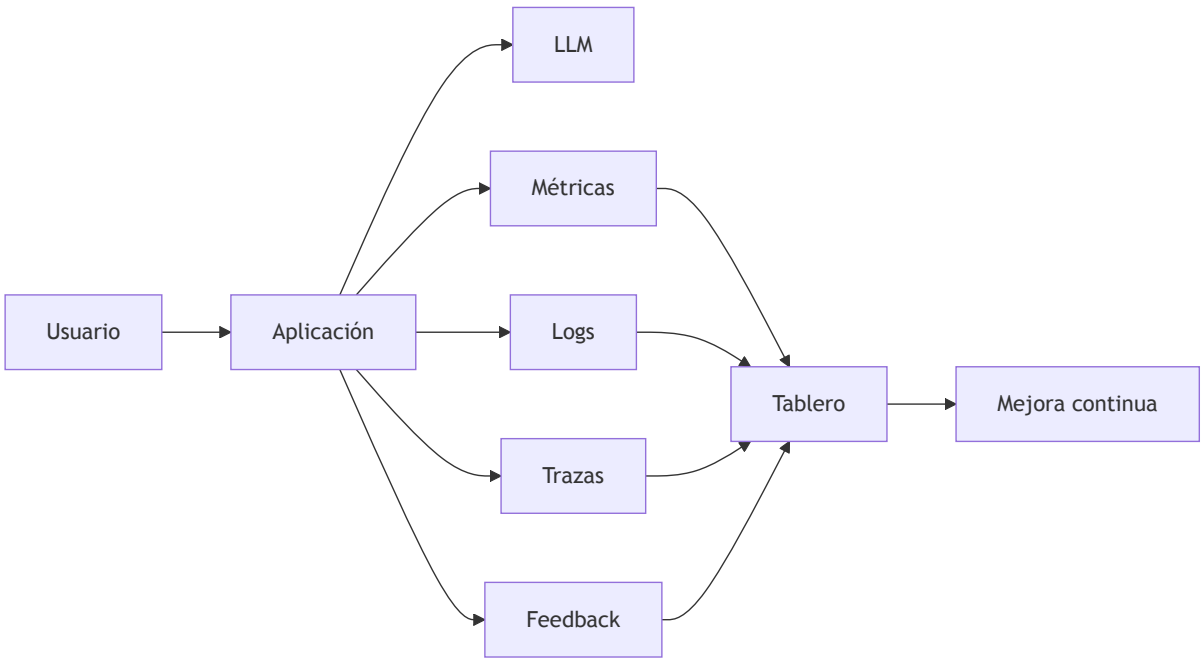
### Estrategia de despliegue

Se recomienda realizar la puesta en producción de manera gradual.

| Etapa              |  | Objetivo                                                     |
|--------------------|--|--------------------------------------------------------------|
| Piloto             |  | Validar el comportamiento con un grupo reducido de usuarios. |
| Despliegue parcial |  | Incrementar progresivamente el alcance.                      |
| Producción         |  | Habilitar el servicio para toda la organización.             |
| Optimización       |  | Ajustar el sistema utilizando métricas reales.               |

Cada etapa debe contar con criterios claros para avanzar o retroceder. En un sistema basado en LLM, retroceder no implica revertir el modelo subyacente: lo que puede restaurarse es el prompt activo, la configuración del orquestador o los parámetros de observabilidad, lo que permite volver a un estado funcional anterior con rapidez y sin riesgo de pérdida estructural.

## Observabilidad



La observabilidad permite comprender cómo se comporta el sistema y detectar oportunidades de mejora antes de que se conviertan en incidentes.

## Métricas recomendadas

Entre los indicadores más útiles se encuentran:

- tiempo medio de respuesta;
- costo por interacción;
- tasa de éxito de las tareas;
- porcentaje de respuestas corregidas por usuarios;
- frecuencia de uso por funcionalidad;
- incidentes detectados en producción.

Las métricas deben analizarse de forma periódica y compararse entre versiones.

## Mejora continua

Una solución madura evoluciona mediante un ciclo permanente.

1. Observar el comportamiento.
2. Detectar oportunidades de mejora.
3. Priorizar cambios.
4. Implementar una nueva versión.
5. Ejecutar nuevamente los *Evaluation Sets*.
6. Desplegar de forma controlada.

Este ciclo convierte el aprendizaje obtenido en producción en conocimiento reutilizable para futuras versiones.

## Caso de estudio

Tras desplegar el asistente corporativo, el equipo observa que muchas consultas requieren una aclaración adicional antes de generar una respuesta útil. Ese dato operativo —la alta frecuencia de turnos de clarificación previos a una respuesta efectiva— llevó al equipo a incorporar un paso de clarificación explícita en el flujo de interacción, lo que constituyó una decisión de diseño conversacional derivada directamente de la observación en producción.

En lugar de modificar directamente el modelo, el equipo analizó los registros, incorporó nuevos casos al conjunto de evaluación, ajustó el diseño conversacional y publicó una nueva versión.

Las métricas mostraron una reducción del número de aclaraciones y un aumento en la satisfacción de los usuarios.

## Actividades propuestas

1. Definir un plan de despliegue gradual.
2. Seleccionar las métricas que serán monitoreadas.
3. Diseñar un tablero de observabilidad.
4. Establecer un proceso de mejora continua.
5. Documentar el procedimiento de liberación de nuevas versiones.

## Buenas prácticas

- Desplegar de forma incremental.
- Establecer líneas de base métricas antes de introducir cualquier cambio en producción.
- Automatizar la recolección de métricas.
- Mantener trazabilidad entre versiones y resultados.
- Incorporar el feedback de los usuarios al proceso de mejora.

## Errores frecuentes

- Considerar el despliegue como el final del proyecto.
- No monitorear el comportamiento real del sistema.
- Introducir cambios sin evidencia.
- Ignorar los incidentes menores detectados por los usuarios.

## Ideas clave

- El despliegue marca el inicio de la operación, no el final del desarrollo.
- La observabilidad es un componente esencial del AI Engineering.
- La mejora continua transforma datos operativos en decisiones de arquitectura.

## Transición hacia la siguiente sección

En la próxima y última sección del proyecto integrador realizaremos el cierre del Módulo 2, revisando los aprendizajes alcanzados y estableciendo las competencias que servirán de base para el estudio de los modelos fundacionales en el Módulo 3.

## Módulo 2 — Prompt Engineering Profesional

---

# Capítulo 22 — Proyecto Integrador del Módulo 2

---

## Sección 07 — Cierre del Módulo 2

*"La tecnología cambia con rapidez. Los principios de ingeniería permanecen."*

---

## Objetivos de aprendizaje

---

- Consolidar los aprendizajes del Módulo 2.
  - Identificar las competencias desarrolladas durante el proyecto integrador.
  - Comprender la evolución desde el Prompt Engineering hacia el AI Engineering.
  - Preparar la transición al Módulo 3.
-

# Introducción

---

Quien termina este módulo no es el mismo profesional que lo comenzó. El recorrido no consistió en acumular técnicas de escritura; consistió en internalizar una forma de pensar: que el valor de un sistema basado en Large Language Models (LLM) no reside en la instrucción que lo activa, sino en la ingeniería que lo sostiene.

---

# Competencias desarrolladas

Al finalizar este módulo el lector debería ser capaz de:

| Competencia            | Resultado esperado                                      | Sección de referencia |
|------------------------|---------------------------------------------------------|-----------------------|
| Diseñar prompts        | Construir instrucciones claras y reutilizables.         | Sección 02            |
| Evaluar resultados     | Utilizar métricas y <i>Evaluation Sets</i> .            | Sección 03            |
| Diseñar conversaciones | Gestionar estado, contexto y memoria.                   | Sección 04            |
| Definir arquitecturas  | Organizar componentes desacoplados.                     | Sección 05            |
| Operar soluciones      | Incorporar PromptOps, observabilidad y mejora continua. | Sección 06            |

Estas capacidades constituyen la base técnica sobre la cual se apoyarán los módulos siguientes.

## Criterios de evaluación del proyecto

---

El proyecto integrador se evalúa según los siguientes criterios:

- Calidad del diseño arquitectónico.
- Separación de responsabilidades.
- Reutilización de componentes.
- Robustez conversacional.
- Facilidad de mantenimiento.
- Capacidad de evolución.
- Alineación con las necesidades del negocio.

Estos criterios no son exclusivos de este proyecto. Representan los estándares que rigen cualquier solución de AI Engineering diseñada para operar en producción.

---

## Del Prompt Engineering al AI Engineering

---



La evolución presentada a lo largo del módulo refleja un cambio de perspectiva.

El foco deja de estar en la calidad de una instrucción individual y pasa a concentrarse en el diseño integral de una solución.

---

## Reflexión final

---

El éxito de un proyecto basado en Inteligencia Artificial no depende únicamente del modelo seleccionado.

También depende de:

- comprender el problema del negocio;
- diseñar una arquitectura sostenible;
- medir objetivamente los resultados;
- incorporar procesos de mejora continua;
- trabajar de manera disciplinada durante todo el ciclo de vida.

Estos principios continuarán vigentes independientemente de la evolución de los modelos fundacionales.

---

## Caso de estudio

---

Dos organizaciones utilizan exactamente el mismo modelo de lenguaje.

La primera obtiene resultados irregulares porque desarrolla prompts de forma aislada y sin procesos de evaluación.

La segunda aplica prácticas de Prompt Engineering, arquitectura modular, observabilidad y mejora continua.

Aunque ambas parten de la misma tecnología, la segunda logra una plataforma más estable, escalable y alineada con las necesidades del negocio.

La diferencia no reside en el modelo, sino en la ingeniería aplicada.

---

## Buenas prácticas

---

- Pensar en sistemas completos y no en prompts individuales.
  - Documentar decisiones arquitectónicas.
  - Medir antes de optimizar.
  - Mantener procesos de mejora continua.
  - Diseñar soluciones preparadas para evolucionar.
-

## Errores frecuentes

---

- Creer que el modelo resuelve por sí solo el problema.
  - Descuidar la arquitectura y la operación.
  - Optimizar sin evidencia.
  - Considerar terminado un sistema después del primer despliegue.
-

## Ideas clave

---

- El Prompt Engineering forma parte del AI Engineering.
  - La arquitectura y el proceso son tan importantes como el modelo.
  - La mejora continua constituye una capacidad permanente.
-

## Transición hacia el Módulo 3

---

En el **Módulo 3 — Modelos Fundacionales**, cambiaremos nuevamente el enfoque.

Hasta ahora aprendimos **cómo construir** soluciones con modelos de lenguaje.

A partir del próximo módulo estudiaremos **con qué modelo conviene construirlas**, comparando GPT, Claude, Gemini, Llama, Qwen, Gemma, Mistral, DeepSeek, Phi y otras familias desde una perspectiva técnica, arquitectónica y empresarial. El alcance no se limita a los modelos de lenguaje: el Módulo 3 aborda los modelos fundacionales en sentido amplio, incluyendo modelos de imagen, audio y multimodales.

Con ello comenzará una nueva etapa del libro, orientada a seleccionar la tecnología adecuada para cada escenario de AI Engineering.

---

*"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."*